

Q1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

1. Theoretical Background

A functional dependency (FD) $X \rightarrow Y$ means that the value of attribute set X uniquely determines the value of attribute set Y — for any two tuples that agree on X , they must also agree on Y . FDs are the foundation of relational database design: they express the real-world business rules that a schema must respect (an employee ID determines an employee's name; a course code determines its title), and they are what every normalization technique — 1NF through 5NF — is built around detecting and removing redundancy from.

A candidate key is a minimal set of attributes whose closure (under the given FDs) covers every attribute in the relation. 'Minimal' is important: if any attribute can be removed from the set and the closure still covers the whole relation, the original set was not a candidate key, merely a superkey. A relation may have more than one candidate key; the one chosen for implementation is called the primary key.

2. Identifying the Functional Dependencies

Looking at the semantics of each attribute in STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade):

- SID determines SName — a given student ID always belongs to exactly one student name.
- CID determines CName and Instructor — a given course ID always has exactly one course title and is taught by exactly one instructor (in this simplified schema).
- The combination (SID, CID) determines Grade — a grade is meaningless without specifying both which student and which course it refers to; the same student has different grades in different courses, and the same course has different grades for different students.

Formally, the FD set is:

SID $\rightarrow$ **SName**

CID $\rightarrow$ **CName, Instructor**

SID, CID $\rightarrow$ **Grade**

3. Determining the Candidate Key

To find the candidate key, we compute the attribute closure of {SID, CID} — the only combination that plausibly identifies a row uniquely, since neither SID nor CID alone can determine Grade.

Step	Closure so far	FD Applied
Start	{SID, CID}	—
Apply SID $\rightarrow$ SName	{SID, CID, SName}	SID $\rightarrow$ SName
Apply CID $\rightarrow$ CName, Instructor	{SID, CID, SName, CName, Instructor}	CID $\rightarrow$ CName, Instructor
Apply SID, CID $\rightarrow$ Grade	{SID, CID, SName, CName, Instructor, Grade}	SID, CID $\rightarrow$ Grade

The closure of {SID, CID} reaches all six attributes of the relation, so {SID, CID} is a superkey. Checking minimality: {SID}⁺ = {SID, SName} (does not reach CName, Instructor, Grade), and {CID}⁺ = {CID, CName, Instructor} (does not reach SName, Grade). Neither subset alone is sufficient, so {SID, CID} cannot be reduced further.

4. Conclusion

Candidate Key = {SID, CID}. Since this is the only minimal attribute combination whose closure covers the relation, it is also the sole candidate key and therefore the primary key of STUDENT_COURSE. All non-key attributes (SName, CName, Instructor, Grade) are functionally dependent on this key, but note that SName depends only on SID (part of the key) and CName/Instructor depend only on CID (the other part) — this partial dependency is precisely what Q3's 2NF analysis is designed to catch and remove.

Q2. Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies

1. Theoretical Background

First Normal Form (1NF) is the baseline requirement for any relation to be considered 'relational' at all. A relation is in 1NF if every attribute holds a single, atomic (indivisible) value — no repeating groups, no multi-valued attributes, and no nested tables within a cell. An unnormalized relation (UNF) is one that violates this: it may store a list, a set, or a nested structure inside a single cell, which breaks the fundamental assumption relational algebra and SQL are built on — that a cell holds exactly one value.

2. Example of an Unnormalized Relation

Consider a student enrollment record where the Courses column stores multiple course codes for a single student in one cell:

SID	SName	Courses
1	Ravi	DBMS, OS, CN
2	Meena	DBMS
3	Suresh	OS, DSA

This violates 1NF because the Courses attribute is not atomic — a single cell contains a comma-separated list rather than one value. Practically, this makes the data extremely difficult to query correctly: finding 'all students enrolled in OS' would require string-parsing every row rather than a simple equality filter, and adding or removing a single course means rewriting the entire cell.

3. Step-by-Step Conversion to 1NF

1. Identify the non-atomic attribute — here, Courses, which holds a set of values instead of one.
2. Determine the correct grain of a row. For this data, the natural grain is one row per (student, course) combination — not one row per student.
3. Flatten the repeating group by creating one row for every value that was packed into the multi-valued cell, repeating the other attributes (SID, SName) as needed.
4. Verify atomicity: after flattening, every cell must hold exactly one value with no internal structure.

4. Resulting 1NF Relation

SID	SName	Course
1	Ravi	DBMS
1	Ravi	OS
1	Ravi	CN
2	Meena	DBMS
3	Suresh	OS
3	Suresh	DSA

Every cell now holds a single atomic value, satisfying 1NF. Note the trade-off: SName is now repeated for every course a student takes (Ravi appears three times). This redundancy is expected at the 1NF stage — it is exactly the kind of redundancy that 2NF and 3NF are designed to remove in subsequent decomposition steps, since SName depends only on SID and not on the (SID, Course) pair as a whole.

```
mysql> INSERT INTO Student_UNF VALUES
-> (1,'Ravi','DBMS,OS,CN'), (2,'Meena','DBMS'), (3,'Suresh','OS,DSA');
Query OK, 3 rows affected (0.006 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Student_UNF;
```

```
+-----+-----+-----+
| SID | SName | Courses |
+-----+-----+-----+
| 1 | Ravi | DBMS,OS,CN |
| 2 | Meena | DBMS |
| 3 | Suresh | OS,DSA |
+-----+-----+-----+
3 rows in set (0.000 sec)
```

```
mysql>
```

```
mysql> -- Step 2: The correct 1NF table
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> CREATE TABLE Student_1NF (
->     SID INT, SName VARCHAR(50), Course VARCHAR(20),
->     PRIMARY KEY (SID, Course)
-> );
Query OK, 0 rows affected (0.007 sec)
```

```
mysql> INSERT INTO Student_1NF VALUES
-> (1,'Ravi','DBMS'),(1,'Ravi','OS'),(1,'Ravi','CN'),
-> (2,'Meena','DBMS'),(3,'Suresh','OS'),(3,'Suresh','DSA');
Query OK, 6 rows affected (0.002 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Student_1NF WHERE Course = 'OS';
```

```
+-----+-----+-----+
| SID | SName | Course |
+-----+-----+-----+
| 1 | Ravi | OS |
| 3 | Suresh | OS |
+-----+-----+-----+
2 rows in set (0.000 sec)
```

Q3. Given relation $R(A,B,C,D,E)$ with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies

1. Theoretical Background

Second Normal Form (2NF) builds on 1NF by additionally requiring that every non-prime attribute (an attribute that is not part of any candidate key) be fully functionally dependent on the whole of every candidate key — not just part of it. A partial dependency occurs specifically when the candidate key is composite (made of two or more attributes) and some non-prime attribute depends on only a subset of that key. If every candidate key in a relation is a single attribute, the relation is automatically in 2NF, since a single-attribute key cannot be partially depended upon.

2. Finding the Candidate Key of $R(A,B,C,D,E)$

Given FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. We compute the closure of $\{A\}$:

Step	Closure	FD Applied
Start	$\{A\}$	—
Apply $A \rightarrow BC$	$\{A,B,C\}$	$A \rightarrow BC$
Apply $BC \rightarrow D$	$\{A,B,C,D\}$	$BC \rightarrow D$
Apply $D \rightarrow E$	$\{A,B,C,D,E\}$	$D \rightarrow E$

$\{A\}^+ = \{A,B,C,D,E\}$ = all attributes, so $\{A\}$ alone is a candidate key. This is significant: since the key is a single attribute, no partial dependency can exist by definition — partial dependency requires a multi-attribute key with a non-prime attribute depending on a proper subset of it. R is therefore already trivially in 2NF as given.

3. Why the Question Still Asks for Decomposition

Even though no partial dependency violation exists, the FD chain $A \rightarrow BC \rightarrow D \rightarrow E$ reveals a transitive structure (A determines BC , which determines D , which determines E) that is worth decomposing along, both as a demonstration of following FDs correctly and as groundwork toward 3NF/BCNF, where this same relation would in fact need decomposition to remove the transitive dependency $A \rightarrow D$ and $A \rightarrow E$ (both flowing through non-key intermediate attributes).

4. Decomposition Following the FD Structure

Decomposing along each FD's determinant and dependents:

Sub-relation	Attributes	Key	Source FD
R1	A, B, C	A	$A \rightarrow BC$
R2	B, C, D	B, C	$BC \rightarrow D$
R3	D, E	D	$D \rightarrow E$

Each sub-relation's key (A in R1, BC in R2, D in R3) is a single or fully-covering determinant for that relation, so every non-key attribute is fully dependent on its own relation's key — 2NF is satisfied throughout, and the decomposition is lossless because $R1 \bowtie R2 \bowtie R3$ reconstructs R exactly (each join uses a common attribute — A/BC pairing through B,C, and D pairing through D — that also functionally determines the next relation's remaining attributes).

```

mysql> CREATE TABLE R1 (A INT PRIMARY KEY, B INT, C INT);
Query OK, 0 rows affected (0.016 sec)

mysql> CREATE TABLE R2 (B INT, C INT, D INT, PRIMARY KEY (B,C));
Query OK, 0 rows affected (0.008 sec)

mysql> CREATE TABLE R3 (D INT PRIMARY KEY, E INT);
Query OK, 0 rows affected (0.006 sec)

mysql>
mysql> INSERT INTO R1 VALUES (1,10,100),(2,20,200);
Query OK, 2 rows affected (0.001 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO R2 VALUES (10,100,1000),(20,200,2000);
Query OK, 2 rows affected (0.001 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO R3 VALUES (1000,'X'),(2000,'Y');
ERROR 1366 (HY000): Incorrect integer value: 'X' for column 'E' at row 1
mysql> INSERT INTO R3 VALUES (1000,1),(2000,2);
Query OK, 2 rows affected (0.008 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT R1.A, R1.B, R1.C, R2.D, R3.E
-> FROM R1
-> JOIN R2 ON R1.B = R2.B AND R1.C = R2.C
-> JOIN R3 ON R2.D = R3.D;
+---+---+---+---+---+
| A | B   | C   | D   | E   |
+---+---+---+---+---+
| 1 | 10  | 100 | 1000 | 1   |
| 2 | 20  | 200 | 2000 | 2   |
+---+---+---+---+---+
2 rows in set (0.009 sec)

```

Q4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

1. Theoretical Background

Third Normal Form (3NF) requires that a relation already be in 2NF, and additionally that no non-prime attribute be transitively dependent on a candidate key. A transitive dependency exists when $X \rightarrow Y$ and $Y \rightarrow Z$ hold, where Y is not a candidate key (and X is), which implies $X \rightarrow Z$ indirectly through Y rather than directly. This matters because Z 's value is effectively controlled by Y , not by X — updating Y should logically update Z everywhere, but if Z is stored redundantly alongside every X that happens to relate to that Y , inconsistency becomes possible.

2. Identifying the Transitive Dependency

In EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID), the natural FDs are:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Here EmpID is the candidate key (EmpID^+ reaches all attributes: $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$).

But observe the chain $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$ and $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID}$: DeptName and ManagerID are determined by DeptID, which is itself just a non-key attribute of EMPLOYEE (not the key). This is a textbook transitive dependency — EmpID determines DeptName and ManagerID only indirectly, through DeptID.

3. Why This Causes Problems

- Redundancy: DeptName and ManagerID repeat for every employee in the same department.
- Update anomaly: changing a department's manager requires updating every employee row in that department; missing even one row leaves the database inconsistent.
- Insertion anomaly: a new department with its manager cannot be recorded until at least one employee is assigned to it.
- Deletion anomaly: removing the last employee in a department silently deletes all knowledge of that department's name and manager.

4. Decomposition to 3NF

The transitive dependency is removed by splitting the relation along the $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$ dependency, separating department-level facts from employee-level facts:

Relation	Attributes	Key	Removes
EMP	EmpID, EmpName, DeptID	EmpID	—
DEPT	DeptID, DeptName, ManagerID	DeptID	Transitive dependency

Now DeptName and ManagerID depend directly on DeptID (the key of DEPT), and EmpName depends directly on EmpID (the key of EMP) — every non-key attribute depends on 'the key, the whole key, and nothing but the key,' which is the informal definition of 3NF. The two relations can always be rejoined via DeptID with no information loss.

```
mysql> SELECT * FROM EMPLOYEE_UNNORM;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| 1     | Arun    | 10     | IT       | 100       |
| 2     | Bala    | 10     | IT       | 100       |
| 3     | Chitra  | 20     | HR       | 200       |
+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

```
mysql> UPDATE DEPT SET ManagerID = 'M999' WHERE DeptID = 10;
```

```
Query OK, 1 row affected (0.012 sec)
```

```
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT E.EmpName, D.DeptName, D.ManagerID FROM EMP E JOIN DEPT D ON E.DeptID = D.DeptID;
```

EmpName	DeptName	ManagerID
Arun	IT	M999
Bala	IT	M999
Chitra	HR	M200

```
3 rows in set (0.000 sec)
```


Q5. Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

1. Theoretical Background

Boyce-Codd Normal Form (BCNF) is a stricter version of 3NF. A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey — meaning the left-hand side of every FD must be able to determine every attribute in the relation on its own. 3NF permits a narrow exception (a determinant that is not a superkey is tolerated if every attribute on the right is itself part of some candidate key), but BCNF does not: any FD whose left side isn't a superkey is a violation, full stop. This makes BCNF the form that most completely eliminates redundancy caused by functional dependencies, at the occasional cost of not preserving every original FD after decomposition.

2. Step 1 — Find All Candidate Keys

Given FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$. Testing $\{A,B\}$:

Step	Closure	FD Applied
Start	$\{A,B\}$	—
Apply $AB \rightarrow C$	$\{A,B,C\}$	$AB \rightarrow C$
Apply $C \rightarrow D$	$\{A,B,C,D\}$	$C \rightarrow D$

$\{A,B\}^+ = \{A,B,C,D\}$ = all attributes, so AB is a superkey, and since neither A nor B alone determines anything new, $\{A,B\}$ is a candidate key.

3. Step 2 — Check Every FD Against the BCNF Rule

FD	Is LHS a superkey?	Verdict
$AB \rightarrow C$	Yes — AB is the candidate key	Satisfies BCNF
$C \rightarrow D$	No — $C^+ = \{A,C,D\}$, missing B	VIOLATES BCNF
$D \rightarrow A$	No — $D^+ = \{A,D\}$, missing B,C	VIOLATES BCNF

R is NOT in BCNF, because both $C \rightarrow D$ and $D \rightarrow A$ have determinants that are not superkeys.

4. Step 3 — Decompose Using the Violating FD

We decompose using $C \rightarrow D$ (the most direct violation). This splits R into a relation holding the dependency itself, and a relation holding everything else plus the determinant (to allow lossless rejoining):

Relation	Attributes	Key	BCNF Check
R_1	C, D	C	$C \rightarrow D$ holds, C is the key $\rightarrow$ BCNF
R_2	A, B, C	A, B	$AB \rightarrow C$ holds, AB is the key $\rightarrow$ BCNF

Both R_1 and R_2 are now individually in BCNF. Note the trade-off: the original FD $D \rightarrow A$ is no longer directly enforceable as a constraint within either sub-relation (it can only be checked by joining R_1 and R_2 back together). This is an accepted and well-known limitation of BCNF decomposition — it always guarantees a lossless join, but does not always guarantee dependency preservation, unlike 3NF decomposition, which guarantees both.

```

mysql> CREATE TABLE R1_CD (C INT PRIMARY KEY, D INT);
Query OK, 0 rows affected (0.017 sec)

mysql> CREATE TABLE R2_ABC (A INT, B INT, C INT, PRIMARY KEY (A,B));
Query OK, 0 rows affected (0.007 sec)

mysql>
mysql> INSERT INTO R1_CD VALUES (100,'X1'),(200,'X2');
ERROR 1366 (HY000): Incorrect integer value: 'X1' for column 'D' at row 1
mysql> INSERT INTO R2_ABC VALUES (1,1,100),(2,2,200),(3,3,100);
Query OK, 3 rows affected (0.010 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO R1_CD VALUES (100,1),(200,2);
Query OK, 2 rows affected (0.008 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO R2_ABC VALUES (1,1,100),(2,2,200),(3,3,100);
ERROR 1062 (23000): Duplicate entry '1-1' for key 'R2_ABC.PRIMARY'
mysql> SELECT R2.A, R2.B, R2.C, R1.D
        -> FROM R2_ABC R2 JOIN R1_CD R1 ON R2.C = R1.C;
+---+---+---+---+
| A | B | C   | D   |
+---+---+---+---+
| 1 | 1 | 100 | 1   |
| 2 | 2 | 200 | 2   |
| 3 | 3 | 100 | 1   |
+---+---+---+---+
3 rows in set (0.007 sec)

```

Q6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

1. Theoretical Background

When a relation is decomposed into two or more sub-relations, the decomposition is 'lossless-join' if joining the sub-relations back together reproduces exactly the original relation — no spurious tuples, no lost information. The tabular (Chase) algorithm is a formal, mechanical procedure for testing this: it builds a tableau (a small table representing the decomposition symbolically), then repeatedly applies the given FDs to force matching symbols into agreement. If this process ever produces a row made entirely of unsubscripted ('a') symbols, the decomposition is proven lossless.

2. Example Setup

Let $R(A,B,C,D)$ be decomposed into $R_1(A,B,C)$ and $R_2(C,D)$, given the FD $C \rightarrow D$.

3. Step 1 — Build the Initial Tableau

Each row represents one sub-relation. Attributes present in that sub-relation get an unsubscripted symbol (a_1, a_2, a_3, a_4 for columns A,B,C,D respectively); attributes absent from that sub-relation get a subscripted symbol unique to that row (b_{ij} , row i , column j).

	A	B	C	D
Row 1 (R1: A,B,C)	a_1	a_2	a_3	b_{14}
Row 2 (R2: C,D)	b_{21}	b_{22}	a_3	a_4

4. Step 2 — Apply Each FD

Apply $C \rightarrow D$: find all rows that agree on column C. Row 1 and Row 2 both show a_3 in column C — they agree. Therefore their D-values must be forced equal. Row 2's D-value (a_4) is unsubscripted, so it 'wins' and Row 1's D-value (b_{14}) is changed to a_4 .

	A	B	C	D
Row 1 (updated)	a_1	a_2	a_3	a_4
Row 2	b_{21}	b_{22}	a_3	a_4

5. Step 3 — Check for an All-'a' Row

Row 1 now reads (a_1, a_2, a_3, a_4) — entirely unsubscripted symbols. Under the Chase algorithm's stopping rule, the appearance of any fully-unsubscripted row proves that the decomposition can reconstruct at least one full original tuple exactly, which is the formal condition for a lossless-join decomposition.

6. Conclusion

The decomposition $R_1(A,B,C), R_2(C,D)$ with FD $C \rightarrow D$ is LOSSLESS. This matches the well-known theorem that a two-way decomposition R_1, R_2 of R is lossless if and only if the common attributes ($R_1 \cap R_2$) functionally determine either R_1 or R_2 in full — here, C (the common attribute) determines D , which is exactly $(R_1 \cap R_2) \rightarrow (R_2 - R_1)$, satisfying the theorem directly.

```
mysql> CREATE TABLE R1_chase (A INT, B INT, C INT);
Query OK, 0 rows affected (0.010 sec)

mysql> CREATE TABLE R2_chase (C INT PRIMARY KEY, D VARCHAR(5));
Query OK, 0 rows affected (0.006 sec)

mysql> INSERT INTO R1_chase SELECT A,B,C FROM R_ORIGINAL;
Query OK, 0 rows affected (0.002 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> INSERT INTO R2_chase SELECT DISTINCT C,D FROM R_ORIGINAL;
Query OK, 0 rows affected (0.008 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT R1.A, R1.B, R1.C, R2.D
      -> FROM R1_chase R1 JOIN R2_chase R2 ON R1.C = R2.C;
Empty set (0.007 sec)
```

Q7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

1. Theoretical Background

A multi-valued dependency (MVD), written $X \twoheadrightarrow Y$, exists when, for a fixed value of X , the set of Y -values associated with it is independent of the values of any other attribute Z in the relation. MVDs typically arise when a relation tries to record two independent multi-valued facts about the same entity within a single table — forcing every combination of the two independent facts to be listed out, purely as a side effect of both being attached to the same key, not because the two facts are actually related to each other. Fourth Normal Form (4NF) requires a relation to be in BCNF and additionally have no non-trivial MVDs other than ones where the left-hand side is a superkey.

2. Identifying the MVDs in TEACHER(TID, Subject, Hobby)

A teacher may teach several subjects and independently have several hobbies — a teacher's hobbies have no logical connection to which subjects they teach. Yet if both facts are stored in one relation, every subject must be paired with every hobby for that teacher, purely to keep the table 'complete':

TID	Subject	Hobby
T1	Math	Chess
T1	Math	Painting
T1	Physics	Chess
T1	Physics	Painting

This is the signature of an MVD problem: T1 teaches 2 subjects and has 2 hobbies, but the table requires $2 \times 2 = 4$ rows to represent this — pure redundancy caused entirely by the Cartesian pairing, not by any real relationship between Subject and Hobby.

$TID \twoheadrightarrow Subject$

$TID \twoheadrightarrow Hobby$

3. Why This Violates 4NF

TEACHER may already be in BCNF (there are no non-trivial single-valued FDs to violate — no attribute functionally determines another in the traditional sense), yet it still exhibits severe redundancy. This is precisely the gap 4NF exists to close: BCNF only reasons about functional dependencies, but says nothing about the independent multi-valued facts problem shown above.

4. Decomposition to 4NF

The fix is to separate each independent multi-valued fact into its own relation:

Relation	Attributes	Captures
TEACHER_SUBJECT	TID, Subject	$TID \twoheadrightarrow Subject$
TEACHER_HOBBY	TID, Hobby	$TID \twoheadrightarrow Hobby$
TID	Subject	
T1	Math	
T1	Physics	
TID	Hobby	
T1	Chess	
T1	Painting	

Now each relation stores only one independent fact per row (2 rows + 2 rows = 4 rows total, versus the original 4-row Cartesian-product table) — for a teacher with 5 subjects and 5 hobbies, this difference would be 10 rows instead of 25, and the redundancy only grows worse as either list grows in the un-decomposed version.

```
mysql> CREATE TABLE TEACHER_BEFORE (TID VARCHAR(5), Subject VARCHAR(20), Hobby VARCHAR(20));
Query OK, 0 rows affected (0.015 sec)

mysql> INSERT INTO TEACHER_BEFORE VALUES
    -> ('T1','Math','Chess'),('T1','Math','Painting'),
    -> ('T1','Physics','Chess'),('T1','Physics','Painting');
Query OK, 4 rows affected (0.002 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT COUNT(*) AS RowCount FROM TEACHER_BEFORE;
+-----+
| RowCount |
+-----+
|         4 |
+-----+
1 row in set (0.009 sec)
```

Q8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

1. Theoretical Background

A join dependency (JD) is the most general form of constraint in relational theory, and it is what 5NF (also called Project-Join Normal Form, PJNF) is built to address. A relation R satisfies a join dependency $\Join(R_1, R_2, \dots, R_n)$ if R can always be reconstructed exactly by projecting it onto $R_1, R_2, \dots, R_n$ and then joining those projections back together. A relation can satisfy such a join dependency without that dependency being implied by any of its candidate keys — and when that happens, the relation suffers redundancy that not even BCNF or 4NF can detect, because both of those forms only reason about two-way (binary) relationships, not genuinely three-way (or higher) ones.

2. The Classic Example — Supplier / Part / Project

Consider SPJ(S, P, J) recording that supplier S supplies part P to project J , governed by the following real-world business rule: 'If supplier S supplies part P , AND part P is used in project J , AND supplier S supplies to project J at all — then supplier S must supply part P specifically to project J .' This is a three-way constraint that cannot be expressed as a simple key or a pair of binary FDs/MVDs.

S	P	J
S1	P1	J1
S1	P2	J1
S1	P1	J2
S1	P2	J2

If S1 supplies both P1 and P2, and both parts are used in both J1 and J2, and S1 supplies both projects, then by the business rule above, all four (S,P,J) combinations shown must exist — this is redundancy driven entirely by the three-way join dependency, not by any simpler FD or MVD. Worse, if any one of the four rows were missing, the table would misrepresent the business rule despite looking 'complete' by ordinary key standards.

3. The Join Dependency

JD: $\Join(\{S,P\}, \{P,J\}, \{S,J\})$

This states that SPJ can always be reconstructed by joining its three binary projections back together — and crucially, this JD is not implied by SPJ's only candidate key $\{S,P,J\}$, which is exactly the condition that makes SPJ fail 5NF despite already satisfying 4NF and BCNF.

4. Decomposition to 5NF

Relation	Attributes	Captures
SP	S, P	Which parts each supplier supplies (in general)
PJ	P, J	Which parts each project uses
SJ	S, J	Which projects each supplier supplies to

$SP \Join PJ \Join SJ$ reconstructs SPJ exactly (this is the join dependency holding by construction), and now each binary relation independently records only a pairwise fact with no redundant three-way repetition — 5NF is achieved, and this is provably the final normal form with respect to join dependencies: no further decomposition based on join dependencies can reduce redundancy any further.

```

mysql> CREATE TABLE SP (S VARCHAR(5), P VARCHAR(5), PRIMARY KEY(S,P));
Query OK, 0 rows affected (0.018 sec)

mysql> CREATE TABLE PJ (P VARCHAR(5), J VARCHAR(5), PRIMARY KEY(P,J));
Query OK, 0 rows affected (0.007 sec)

mysql> CREATE TABLE SJ (S VARCHAR(5), J VARCHAR(5), PRIMARY KEY(S,J));
Query OK, 0 rows affected (0.007 sec)

mysql>
mysql> INSERT INTO SP VALUES ('S1','P1'),('S1','P2');
Query OK, 2 rows affected (0.004 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO PJ VALUES ('P1','J1'),('P1','J2'),('P2','J1'),('P2','J2');
Query OK, 4 rows affected (0.002 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO SJ VALUES ('S1','J1'),('S1','J2');
Query OK, 2 rows affected (0.001 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Reconstruct the original SPJ relation via the three-way join
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT SP.S, SP.P, PJ.J
      -> FROM SP
      -> JOIN PJ ON SP.P = PJ.P
      -> JOIN SJ ON SP.S = SJ.S AND PJ.J = SJ.J;
+-----+-----+-----+
| S   | P   | J   |
+-----+-----+-----+
| S1  | P1  | J1  |
| S1  | P1  | J2  |
| S1  | P2  | J1  |
| S1  | P2  | J2  |
+-----+-----+-----+
4 rows in set (0.007 sec)

```


Q9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

1. Theoretical Background

Anomalies are the practical, real-world symptoms of an under-normalized schema — they are the reason normalization matters in the first place, beyond being a theoretical exercise. The three classic anomaly types are: an insertion anomaly (you cannot record a fact until an unrelated fact is also available), a deletion anomaly (deleting one fact accidentally destroys another, unrelated fact), and an update anomaly (a single real-world fact is stored redundantly in multiple rows, so updating it correctly requires touching every copy, and missing even one produces inconsistent data).

2. The Poorly Designed College Schema

Consider a single flat relation intended to capture which students are enrolled in which courses, taught in which departments:

COLLEGE(StudentID, StudentName, CourseID, CourseName, DeptID, DeptName, HOD)

StudentID	StudentName	CourseID	CourseName	DeptID	DeptName	HOD
S1	Ravi	C101	DBMS	D1	CSE	Dr. Menon
S1	Ravi	C102	OS	D1	CSE	Dr. Menon
S2	Meena	C101	DBMS	D1	CSE	Dr. Menon

3. Identifying the Anomalies

Insertion Anomaly

A new course cannot be added to the catalog — along with its department and HOD — until at least one student enrolls in it, since CourseID/CourseName/DeptID only exist as attributes of a row that also requires a StudentID. This artificially couples two unrelated facts (a course existing, and a student taking it).

Deletion Anomaly

If S2 is the only student enrolled in course C101 and that row is deleted (e.g., S2 drops the course), all information about C101 — its name, its department, and even that department's HOD — is lost from the database, purely as a side effect of removing an unrelated enrollment fact.

Update Anomaly

If the CSE department's HOD changes from Dr. Menon to a new appointee, every row belonging to a CSE student and course must be updated individually. In a college with thousands of students, this could mean thousands of UPDATE operations for what is conceptually a single fact — and if even one row is missed, the database now contains two different HOD values for the same department simultaneously.

4. Functional Dependencies

StudentID $\rightarrow$ StudentName

CourseID $\rightarrow$ CourseName, DeptID

DeptID $\rightarrow$ DeptName, HOD

Note the transitive chain CourseID $\rightarrow$ DeptID $\rightarrow$ DeptName, HOD, which is the root cause of the anomalies above — exactly the same transitive-dependency pattern analyzed in Q4.

5. Normalization to 3NF

Relation	Attributes	Key
STUDENT	StudentID, StudentName	StudentID
COURSE	CourseID, CourseName, DeptID	CourseID
DEPARTMENT	DeptID, DeptName, HOD	DeptID
ENROLLMENT	StudentID, CourseID	StudentID, CourseID

This resolves all three anomalies: a course can now be added to COURSE independently of any enrollment (fixes insertion); dropping an ENROLLMENT row no longer touches COURSE or DEPARTMENT data (fixes deletion); and a department's HOD is stored exactly once in DEPARTMENT, so updating it is a single-row operation (fixes update).

Q10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

1. Theoretical Background

The closure of an attribute set X (written X^+) under a set of functional dependencies F is the complete set of attributes that can be functionally determined starting from X , by repeatedly applying the FDs in F . Closure computation is the core algorithmic tool of relational design: it is used to test whether a given attribute set is a superkey (does its closure cover the whole relation?), to test whether a specific FD is implied by others, and — as this question asks — to systematically discover every candidate key of a relation.

2. Example Setup

Let $R(A,B,C,D,E)$ have FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow E$, $E \rightarrow A$.

3. Step 1 — Note Which Attributes Never Appear on a Right-Hand Side

Scanning the FD set, B never appears on the right-hand side of any dependency. This means nothing in the FD set can ever derive B starting from other attributes — so B must be included in every candidate key, since no combination of the other attributes can ever produce B in its closure.

4. Step 2 — Compute the Closure of $\{A, B\}$

Step	Closure	FD Applied
Start	$\{A,B\}$	—
Apply $AB \rightarrow C$	$\{A,B,C\}$	$AB \rightarrow C$
Apply $C \rightarrow D$	$\{A,B,C,D\}$	$C \rightarrow D$
Apply $D \rightarrow E$	$\{A,B,C,D,E\}$	$D \rightarrow E$

$\{A,B\}^+ = \{A,B,C,D,E\}$ = all attributes of R . So $\{A,B\}$ is a superkey.

5. Step 3 — Verify Minimality

Subset	Closure	Covers all attributes?
$\{A\}$	$\{A\}$	No
$\{B\}$	$\{B\}$	No

Neither $\{A\}$ alone nor $\{B\}$ alone reaches all attributes (B in particular never grows, since nothing determines it), so $\{A,B\}$ cannot be reduced further — it is minimal, and therefore a genuine candidate key.

6. Step 4 — Check for Any Other Candidate Keys

Since B must appear in every candidate key (established in Step 1), any other candidate key would have to be some other set containing B whose closure also reaches all attributes, with no smaller subset doing the same. Testing combinations such as $\{B,C\}$, $\{B,D\}$, $\{B,E\}$ against the same closure process shows each of these closes to the full attribute set too (e.g., $\{B,C\}^+$: $C \rightarrow D$ gives $\{B,C,D\}$, $D \rightarrow E$ gives $\{B,C,D,E\}$, $E \rightarrow A$ gives $\{A,B,C,D,E\}$) — but these are not minimal, because $\{A,B\}$ is already a strict subset reachable indirectly (A can be derived, so keys not built from the true minimal generator are superkeys, not candidate keys, whenever a smaller determining set exists).

Applying minimality rigorously: $\{A,B\}$ is the unique minimal set. $\{B,C\}$, $\{B,D\}$, $\{B,E\}$ are all superkeys but not candidate keys, since each properly contains information redundant with what $\{A,B\}$ already provides via the FD chain.

7. Conclusion

Candidate Key = $\{A, B\}$ (the only minimal candidate key). This single result also lets us classify every attribute: A and B are prime attributes (they belong to the candidate key); C , D , E are non-prime attributes. This classification is itself the starting point for checking 2NF, 3NF, and BCNF on this same relation, since every one of those forms is defined in terms of how non-prime attributes depend on candidate keys.